Chapter 6. The Relevance of Reactive Java in Light of Virtual Threads

> *I think Loom is going to kill reactive programming.... Reactive programming was a transitional technology.*
>
> —Brian Goetz

While I won't comment on the preceding quote—at least not yet—this chapter will introduce an existing alternative that has long been popular among many developers: Reactive Java.

Over the past few chapters, we've explored virtual threads in great detail, understanding how they work, their advantages, and their role in making concurrency in Java more accessible. We recognized that virtual threads are a great innovation in the concurrency landscape. They allow developers to write highly concurrent applications using the familiar imperative programming model while efficiently handling blocking operations.

However, virtual threads are not the only solution for efficiently handling concurrency and blocking I/O. Before virtual threads came into the limelight, many developers turned to reactive programming to build scalable, non-blocking applications. This approach, often associated with Project Reactor, RxJava, Eclipse Vert.x, etc., embraces an entirely different paradigm—one that is event-driven, functional, and inherently asynchronous.

In this chapter, we will explore Reactive Java, examining its execution model, programming style, ease of use and learning curve, challenges, benefits, etc.

Let's begin.

Understanding Reactive Programming in Java

Reactive programming is a declarative paradigm centered on asynchronous data streams and the automatic propagation of change.

Instead of focusing on the step-by-step execution of instructions, developers define the relationships between these streams and how they trans-

form data. These streams can represent anything that changes over time, such as user input, sensor readings, or data from external systems. When a value in a stream changes, the reactive framework automatically propagates that change downstream, updating all dependent components. This allows for efficient handling of asynchronous events and simplifies the development of complex, event-driven applications. Developers can create more maintainable and scalable systems by expressing *what* data transformations and reactions should occur rather than *how* to execute them. Popular reactive programming libraries and frameworks include RxJava, Reactor, and Akka Streams, which provide the tools to create and manipulate these data streams.

In the Java ecosystem, reactive programming is usually associated with *non-blocking, asynchronous operations* and *event-driven architectures*.

REACTIVE SYSTEMS AT A GLANCE

In 2013, the Reactive Manifesto emerged to define what it means to be reactive; it provides a definition of reactive systems and aims to standardize the meaning of *reactive*.

It identifies four key principles:

Responsive

: Reactive systems consistently deliver rapid response times, enhancing user confidence and simplifying error handling.

Resilient

: They maintain responsiveness despite failures by isolating issues, delegating recovery, and using replication for high availability.

Elastic

: They dynamically adapt to varying workloads, efficiently scaling resources up or down to maintain performance.

Message-Driven

: They use asynchronous messaging for loose coupling, isolation, and efficient resource usage, enabling resilience and elasticity.

This essentially comes down to three key aspects: being non-blocking, event-driven, and asynchronous in technical implementation.

Let's unpack these concepts one by one.

Blocking Versus Non-blocking I/O

In the previous chapters, we learned a great deal about blocking operations. Simply put, when an I/O occurs, the caller thread waits until the op-

eration completes. During this time, the thread remains idle, essentially "blocked," until it gets the data it needs. This is what we refer to as a blocking operation.

The alternative approach is non-blocking I/O. In this model, threads are not held up by I/O operations. Instead of waiting, a thread can continue executing other tasks while the I/O happens in the background.

So, how does this work? In a non-blocking system, the operating system takes over the responsibility of handling I/O. When the operation is complete, the OS notifies the application, typically by invoking a callback function. This way, the thread that initiated the request doesn't have to sit idle; it simply gets notified when the data is ready.

To make this idea more concrete, let's walk through an example. Imagine building an HTTP server that needs to handle request pipelining, a critical feature in HTTP/1.1 where clients can send multiple requests without waiting for responses. The server needs to:

- Accept connections from multiple clients
- Process HTTP requests with varying complexity (fast versus slow operations)
- Maintain connection state for keep-alive sessions
- Handle request pipelining correctly
- Scale to handle hundreds or thousands of concurrent connections

That's all there is to it. Now, let's explore how we can implement such a server using both blocking and non-blocking approaches to see the differences in action.

We start with a single-threaded blocking server:

```
public class BlockingHttpServer {
  private static final int PORT = 8080;
  private static final AtomicInteger requestCounter = new AtomicInteger(0);

  public static void main(String[] args) throws IOException {
    System.out.println("Blocking HTTP Server starting on port " + PORT);
    System.out.println("Features: Single-threaded, Request pipelining");
    try (ServerSocket serverSocket = new ServerSocket(PORT)) {
      serverSocket.setReuseAddress(true); ❶
      while (true) {
        Socket clientSocket = serverSocket.accept(); ❷
        handleConnection(clientSocket);
      }
    }
  }

  private static void handleConnection(Socket socket) {
    System.out.println("New connection from: " +
```

```
                        socket.getRemoteSocketAddress());
        try (socket;
             BufferedReader in = new BufferedReader(
                 new InputStreamReader(socket.getInputStream()));
             PrintWriter out = new PrintWriter(
                 socket.getOutputStream(), true)) {
          socket.setSoTimeout(5000); ❸
          boolean keepAlive = true;
          while (keepAlive) {
            HttpRequest request = parseRequest(in); ❹
            if (request == null) {
              break; // Connection closed
            }
            int requestId = requestCounter.incrementAndGet();
            System.out.println("Request #" + requestId + ": " +
                               request.method + " " + request.path);
            // Check for keep-alive
            keepAlive = "keep-alive".equalsIgnoreCase(
                request.getHeader("Connection"));
            // Process request - this may take time! ❺
            processRequest(request);
            // Send response
            sendResponse(out, request, requestId, keepAlive);
          }
        } catch (SocketTimeoutException e) {
          System.out.println("Connection timeout");
        } catch (IOException e) {
          System.err.println("Connection error: " + e.getMessage());
        }
      }
    }
```

Let's examine the key points in this code:

❶ We enable address reuse to avoid "Address already in use" errors during development.

❷ The `accept()` method blocks until a client connects. This is where our single thread gets stuck waiting for new connections.

❸ We set a socket timeout of five seconds to detect dead connections and prevent the server from hanging indefinitely.

❹ The `parseRequest()` method blocks until a complete HTTP request arrives. While waiting for one client's request, the server cannot accept new connections.

❺ This is a critical limitation: if `processRequest()` takes a long time (such as in the event of a slow database query), the entire server blocks. No other clients can connect or be served.

The server requires several helper methods to handle HTTP communication:

```
    private static void sendResponse(PrintWriter out,
                                     HttpRequest request,
                                     int requestId,
                                     boolean keepAlive) {
      // Send HTTP response
      out.println("HTTP/1.1 200 OK");
      out.println("Content-Type: text/plain");
      out.println("Connection: " + (keepAlive ? "keep-alive" : "close"));
      String body = String.format(
          "Request #%d processed\nPath: %s\nTime: %s\n",
          requestId, request.path, Instant.now());
      out.println("Content-Length: " + body.length());
      out.println(); // Empty line between headers and body
      out.print(body);
      out.flush(); ❻
    }
```

❻ Ensures immediate delivery of the HTTP response by forcing any buffered data to be sent to the client, preventing delays in response transmission.

To clearly demonstrate the blocking behavior, we'll simulate different processing times:

```
  static void processRequest(HttpRequest request)
        throws IOException {
      try {
        if (request.path.startsWith("/slow")) {
          Thread.sleep(Duration.of(30, ChronoUnit.SECONDS)); ❼
          System.out.println("  Slow request processed");
        } else if (request.path.startsWith("/medium")) {
          Thread.sleep(Duration.of(500, ChronoUnit.MILLIS));
          System.out.println("  Medium request processed");
        } else {
          Thread.sleep(Duration.of(100, ChronoUnit.MILLIS));
          System.out.println("  Fast request processed");
        }
      } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
      }
    }
```

❼ A slow request blocks the entire server for 30 seconds. During this time, no new connections can be accepted, and no other requests can be processed. This is added to simulate the slowness of the API call.

Our simple `HttpRequest` class captures the essential elements needed for processing:

```
  static class HttpRequest {
    String method;
    String path;
    String version;
    Map<String, String> headers = new HashMap<>();
    String remainingBuffer; // For non-blocking server
    String getHeader(String name) {
      return headers.get(name.toLowerCase());
    }
  }
```

And here is the helper method that parses the request.

```
static HttpRequest parseRequest(BufferedReader in)
      throws IOException {
    // Read request line
    String requestLine = in.readLine(); //
    if (requestLine == null || requestLine.isEmpty()) {
      return null;
    }
    String[] parts = requestLine.split(" ");
    if (parts.length != 3) {
      throw new IOException("Invalid request line");
    }
    HttpRequest request = new HttpRequest();
    request.method = parts[0];
    request.path = parts[1];
    request.version = parts[2];
    // Read headers
    String headerLine;
    while ((headerLine = in.readLine()) != null) { //
      if (headerLine.isEmpty()) {
        break; // End of headers
      }
      int colonPos = headerLine.indexOf(':');
      if (colonPos > 0) {
        String name = headerLine.substring(0, colonPos).trim();
        String value = headerLine.substring(colonPos + 1).trim();
        request.headers.put(name.toLowerCase(), value);
      }
    }
    return request;
  }
```

In this implementation, we created a `ServerSocket` that listens on port `8080`. A `Socket` represents a single endpoint in a two-way communication link between programs running on a network. On the other hand, a `ServerSocket` continuously listens for incoming connections and processes client requests accordingly.

Here's how it works in our case:

- The server waits for a client to connect.
- Once a connection is established, the server hands it off to the `handleConnection()` method, which takes care of the interaction.
- This method processes HTTP requests, handling keep-alive connections and request pipelining. The specifics of HTTP processing showcase how blocking I/O impacts server performance.

However, there's a catch. When the server handles a client, the calling thread is blocked until the client disconnects. If the client stops sending data but remains connected, the server stays stuck, waiting for input. Because it's a single-threaded server, no other clients can connect until the current session ends.

To test this server and observe its blocking behavior, we can use curl. Let's run a few experiments:

Terminal 1: Start the server

```
$ java BlockingHttpServer.java
Single-threaded HTTP Server with Pipelining starting on port 8080
Features: Request pipelining, Keep-alive connections
```

The server is now running and waiting for connections. Let's send a simple request:

Terminal 2: Send a single fast request

```
$ curl -v http://localhost:8080/fast
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
> GET /fast HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Content-Type: text/plain
< Connection: close
< Content-Length: 67
<
Request #1 processed
Path: /fast
Time: 2025-06-14T09:27:02.235713Z
* Closing connection
```

This fast request completes immediately. The server processes it and closes the connection. Now let's demonstrate the blocking problem:

Terminal 3: Test blocking behavior—first send a slow request

```
$ curl -v http://localhost:8080/slow
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
> GET /slow HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Content-Type: text/plain
< Connection: close
< Content-Length: 67
<
Request #2 processed
Path: /slow
Time: 2025-06-14T09:28:16.218432Z
* Closing connection
```

While this slow request is processing, let's try to connect with another client:

Terminal 4: While the slow request is processing, try a fast request

```
$ curl -v --max-time 5 http://localhost:8080/fast
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
> GET /fast HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
* Operation timed out after 5006 milliseconds with 0 bytes received
* Closing connection
curl: (28) Operation timed out after 5006 milliseconds with 0 bytes received
```

Notice how the second client successfully establishes a TCP connection (the OS handles this), but the server doesn't accept it because it's still

blocked while processing the slow request. This is the fundamental limitation of our single-threaded design.

The most revealing test is to use curl's timing features to measure the blocking effect:

```
# Send three pipelined requests: fast, slow, fast


  $ time curl -s -Z http://localhost:8080/fast \
              http://localhost:8080/slow \
              http://localhost:8080/fast
  Request #11 processed
  Path: /fast
  Time: 2025-06-14T09:34:49.986347Z
  Request #12 processed
  Path: /slow
  Time: 2025-06-14T09:35:19.989656Z
  Request #13 processed
  Path: /fast
  Time: 2025-06-14T09:35:20.094539Z
  curl -s -Z http://localhost:8080/fast http://localhost:8080/slow   0.01s user
  0.01s system 0% cpu 47.811 total
```

Although two of the three requests are fast, the total time exceeds 30 seconds because all requests must be processed sequentially. The slow request in the middle blocks everything.

If you attempt to connect to the server from another client while an existing client session is active, you'll notice that the connection is blocked. The server can only handle one client at a time. This is because our implementation is single-threaded—it waits for the current client to disconnect before accepting a new one.

An easy solution to this problem is multithreading. By spawning a new thread for each client, we allow multiple connections to be handled concurrently. Let's modify our code to introduce multithreaded support:

```
  public class MultithreadedHttpServer {
    private static final int PORT = 8080;
    private static final AtomicInteger requestCounter = new AtomicInteger(0);
    private static final int CONNECTION_THREADS = 10;

    public static void main(String[] args) throws IOException {
      System.out.println("Multi-threaded HTTP Server starting on port " + PORT
      System.out.println("Features: Concurrent connections, Request pipelining
      System.out.println("Connection pool size: " + CONNECTION_THREADS);

      try (ServerSocket serverSocket = new ServerSocket(PORT);
           ExecutorService connectionExecutor =
```

```
                Executors.newFixedThreadPool(CONNECTION_THREADS)) { ❶
            serverSocket.setReuseAddress(true);
            while (true) {
              Socket clientSocket = serverSocket.accept(); ❷
              System.out.println("New connection from: " +
                                 clientSocket.getRemoteSocketAddress());

              // Handle each connection in a separate thread
              connectionExecutor.submit(() -> handleConnection(clientSocket)); ❸
            }
          }
        }

        private static void handleConnection(Socket socket) {
          String clientAddr = socket.getRemoteSocketAddress().toString();
          System.out.println("Thread " + Thread.currentThread().getName() +
                             " handling connection from: " + clientAddr);
          // Same request processing logic as BlockingHttpServer
          // but running in a separate thread ❹
          try (socket;
               BufferedReader in = new BufferedReader(
                   new InputStreamReader(socket.getInputStream()));
               PrintWriter out = new PrintWriter(
                   socket.getOutputStream(), true)) {
            socket.setSoTimeout(5000);
            boolean keepAlive = true;
            while (keepAlive) {
              HttpRequest request = parseRequest(in);
              if (request == null) break;
              int requestId = requestCounter.incrementAndGet();
              System.out.println("Thread " + Thread.currentThread().getName() +
                                 " - Request #" + requestId + ": " +
                                 request.method + " " + request.path);
              keepAlive = "keep-alive".equalsIgnoreCase(
                  request.getHeader("Connection"));
              processRequest(request);
              sendResponse(out, request, requestId, keepAlive);
            }
          } catch (Exception e) {
            System.err.println("Connection error from " + clientAddr +
                               ": " + e.getMessage());
          }
        }
      }
```

Key improvements in the multithreaded version:

❶ We create a fixed thread pool with 10 threads to handle connections.

❷ The main thread can immediately return to accepting new connections.

❸ Each connection runs in its thread, allowing concurrent processing.

❹ Individual connections still process requests sequentially (maintaining proper pipelining), but multiple connections can be served simultaneously.

In our multithreaded implementation, we created a fixed thread pool with 10 threads, allowing up to 10 clients to connect simultaneously. We could increase the thread count to handle more clients, but before the advent of virtual threads, this approach had inherent limitations. Platform threads are expensive, and increasing their number beyond a certain point would not yield any better results, as we discussed in Chapter 2.

With virtual threads, however, scaling becomes as simple as changing one line of code:

```
try (ServerSocket serverSocket = new ServerSocket(PORT);
     ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor();
) {}
```

This would allow the server to handle a massive number of concurrent connections with minimal overhead. But that's not our focus right now. Instead, let's explore what alternatives existed before virtual threads.

One powerful approach was new input/output (NIO), which enables high throughput using a minimal number of threads.

Java NIO is a readiness-based, non-blocking API introduced in JDK 1.4 that replaces stream-oriented java.io with buffer-oriented channels (`SocketChannel`, `FileChannel`, etc.) and a `Selector` event loop. Instead of a thread blocking until an operation completes, a non-blocking `read()` or `write()` call returns immediately, often with zero bytes transferred. At the same time, the channel is registered with the selector. The selector sleeps inside `select()` until the operating system reports that one or more channels are ready for I/O; the thread then processes only those channels, enabling thousands of concurrent connections to be serviced by a handful of threads. Combined with direct or memory-mapped buffers, which allow zero-copy data transfers between user and kernel space, NIO delivers high throughput and scalability without the per-connection thread and stack overhead of the classic thread-per-socket model.

JAVA I/O VERSUS NIO: A QUICK COMPARISON

Java offers two distinct approaches to handling I/O: the traditional java.io package and the more modern java.nio (new input/output). The key difference lies in their approach. Java I/O is stream-based and generally blocking, meaning a thread must wait for operations to complete. On the other hand, Java NIO is buffer-based and non-blocking, allowing greater scalability for handling large data transfers and high-performance applications.

For instance, consider reading a file using Java I/O:

```
try (BufferedReader reader
            = new BufferedReader(
                 new FileReader("input.txt"))) {
    String line;
    while ((line = reader.readLine()) != null) {
      System.out.println(line);
    }
  } catch (IOException e) {
    e.printStackTrace();
  }
```

`BufferedReader` reads text from an input stream, line by line, making it efficient and easy to use for sequential file reading.

Now, contrast that with Java NIO's `FileChannel`:

```
try (FileChannel channel = FileChannel.open(
        Path.of("input.txt"),
        StandardOpenOption.READ)) {
        ByteBuffer buffer = ByteBuffer.allocate(1024);
        while (channel.read(buffer) > 0) {
            buffer.flip();
            while (buffer.hasRemaining()) {
                System.out.print((char) buffer.get());
            }
            buffer.clear();
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
```

This version is buffer-driven, meaning data is first loaded into a `ByteBuffer`, manipulated in memory, and only then processed.

Instead of using `ServerSocket`, NIO introduces `ServerSocketChannel`. Similarly, `Socket` is replaced by

`SocketChannel`. Rather than relying on `InputStream` and `OutputStream`, NIO uses buffers. A buffer acts as a temporary storage area, holding a fixed amount of data before it's sent to its destination. This shift in design provides greater flexibility and efficiency when handling I/O operations.

NOTE

Buffers in Java NIO improve performance by minimizing direct interactions with the operating system and enabling efficient data transfer. Unlike traditional I/O, which reads and writes data byte by byte, buffers allow batch processing, which reduces the number of system calls. This is particularly useful in high-performance applications like network servers and file handling.

One of the key efficiency boosters is zero-copy mechanisms, where `ByteBuffer` can work with memory-mapped files or direct buffers (off-heap memory). This eliminates unnecessary copying of data between user space and kernel space, significantly speeding up large file transfers. Additionally, non-blocking I/O with buffers enables scalable, event-driven applications by allowing a single thread to manage multiple connections efficiently.

Another critical component in NIO is the `Selector`, which plays a vital role in managing multiple channels, such as `ServerSocketChannel` and `SocketChannel`, with a single thread. Think of it as an intelligent traffic controller that listens for various I/O events and directs operations accordingly. For instance, when a new client connection is ready to be accepted, the selector signals an `OP_ACCEPT` event. When data is available to be read from a `SocketChannel`, it triggers an `OP_READ` event. Similarly, when a channel is ready to accept outgoing data, the `OP_WRITE` event is fired, and in client mode, a successful connection attempt results in an `OP_CONNECT` event.

The underlying implementation of selectors is system-dependent. On macOS, for example, `KQueueSelectorImpl` leverages the `queue` system call. A more general-purpose implementation, `PollSelectorImpl`, is based on the `poll` system call. The specific mechanism varies depending on the operating system, but the concept remains the same: efficiently managing multiple channels with minimal thread overhead.

Let's examine some code to see how these components work together in practice:

```
public class NonBlockingHttpServer {
    private static final int PORT = 8080;
    private static final AtomicInteger requestCounter = new AtomicInteger(0
    private final ConcurrentLinkedQueue<PendingUpdate> pendingUpdates
                                                = new ConcurrentLinkedQueue<>();
```

```
    public static void main(String[] args) {
        System.out.println("Starting non-blocking NIO server...");
        System.out.println("Features: Single-threaded event loop, " +
                           "Non-blocking I/O, High concurrency");
        try {
            new NonBlockingHttpServer().start();
        } catch (IOException e) {
            System.err.println("Server failed to start: " + e.getMessage())
        }
    }

    private void start() throws IOException {
        try (Selector selector = Selector.open();
             ServerSocketChannel serverChannel = ServerSocketChannel.open()
            serverChannel.bind(new InetSocketAddress(PORT));
            serverChannel.configureBlocking(false); ❶
            serverChannel.register(selector, SelectionKey.OP_ACCEPT);
            System.out.println("Non-blocking HTTP server on port " + PORT);
            // Single-threaded event loop ❷
            while (true) {
                // Process pending updates from async threads
                processPendingUpdates(); ❸

                selector.select(100); ❹
                Set<SelectionKey> selectedKeys = selector.selectedKeys();
                Iterator<SelectionKey> iterator = selectedKeys.iterator();
                while (iterator.hasNext()) {
                    SelectionKey key = iterator.next();
                    iterator.remove();
                    try {
                        if (key.isAcceptable()) {
                            handleAccept(key, selector); ❺
                        } else if (key.isReadable()) {
                            handleRead(key); ❻
                        } else if (key.isWritable()) {
                            handleWrite(key); ❼
                        }
                    } catch (IOException e) {
                        System.err.println("Error handling key: "
                                                + e.getMessage());
                        key.cancel();
                        if (key.channel() != null) {
                            key.channel().close();
                        }
                    }
                }
                // Process any pending requests
                processAllPendingRequests(selector);
            }
        }
    }
}
```

Key improvements in the non-blocking version:

❶ We configure the server channel as non-blocking, essential for NIO operation.

❷ A single thread handles all I/O events through an event loop.

❸ Before processing events, we handle any selector updates queued by async threads. This crucial step prevents race conditions that could crash the server.

❹ The `selector` uses a timeout (100ms) instead of blocking indefinitely. This ensures pending updates from async threads are processed promptly, even if no I/O events occur.

❺ New connections are accepted without blocking the thread.

❻ Data reading is non-blocking; partial reads are handled gracefully.

❼ Data writing is non-blocking; partial writes are queued for later completion.

In the preceding code, we start by creating a `Selector` using `Selector.open()`. Instead of using a traditional constructor, it offers a static factory method, which is the preferred approach. This `Selector` is the core of our server, allowing it to manage multiple client connections efficiently with just a single thread.

Next, we create a `ServerSocketChannel` and bind it to a specific address. It is crucial to configure the channel as non-blocking using `serverSocketChannel.configureBlocking(false)`. Without this, the server would block on every operation, defeating the purpose of using NIO.

Once the `ServerSocketChannel` is set up, we register it with the `Selector`, specifying that we are interested in `accept` events. This means the server will be notified when a new client attempts to connect.

We also created all the handlers here. These classes take responsibility for each event, and we have just separated them.

Inside the main loop, we first call `processPendingUpdates()`, which handles a critical synchronization challenge. Since `SelectionKey` objects are not thread-safe, we cannot modify them directly from async processing threads. Instead, we use a thread-safe queue to communicate between threads:

```
private void processPendingUpdates() {
    PendingUpdate update;
    while ((update = pendingUpdates.poll()) != null) {
        try {
            if (update.key.isValid()) {
                update.key.interestOps(update.key.interestOps()
```

```
                                | SelectionKey.OP_WRITE);
                }
            } catch (Exception e) {
                System.err.println("Error updating key: " + e.getMessage());
            }
        }
    }

    static class PendingUpdate {
        final SelectionKey key;

        PendingUpdate(SelectionKey key) {
            this.key = key;
        }
    }
```

After processing updates, we call the `selector.select(100)`, which blocks for up to 100 milliseconds. The timeout is important: without it, the selector might block indefinitely, preventing timely processing of updates from async threads. As soon as an event is detected or the timeout expires, `select()` returns, allowing the server to process events or pending updates accordingly.

Once an event is detected, we retrieve the selection keys from the selector. Each key represents a specific event for a registered channel. Since multiple events can occur, the server processes them sequentially within a loop. After handling a key, it is removed from the set to prevent reprocessing.

When `key.isAcceptable()` returns `true`, it indicates that a new client is attempting to connect. At this point, we invoke the `handleAccept(key, selector)` method to handle the accept event.

Let's look at the code of the accept handler now:

```
    private void handleAccept(SelectionKey key, Selector selector)
            throws IOException {
        ServerSocketChannel serverChannel = (ServerSocketChannel) key.channel()
        SocketChannel clientChannel = serverChannel.accept(); ❶

        if (clientChannel != null) {
            clientChannel.configureBlocking(false);
            clientChannel.register(selector, SelectionKey.OP_READ,
                                   new ClientState()); ❷

            System.out.println("Accepted connection from: " +
                               clientChannel.getRemoteAddress());
        }
    }

    static class ClientState {
```

```
        ByteBuffer readBuffer = ByteBuffer.allocate(8192);
        StringBuilder requestBuilder = new StringBuilder();
        ConcurrentLinkedQueue<String> responseQueue =
                                            new ConcurrentLinkedQueue<>(); ❸
        Queue<HttpRequest> pendingRequests = new LinkedList<>();
        boolean keepAlive = true;
        boolean isProcessing = false; ❹
    }
```

The accept handler manages new client connections:

❶ The `accept()` call is non-blocking and may return `null` if no connection is ready.

❷ Each client gets a `ClientState` object to track its connection state and partial data.

❸ The response queue uses `ConcurrentLinkedQueue` instead of a regular `LinkedList`. This is crucial because responses will be added from async processing threads, and we need thread-safe operations without explicit synchronization.

❹ The `isProcessing` flag prevents duplicate processing of requests from the same client, avoiding race conditions when multiple events trigger in quick succession.

In the handle method, we first grab hold of the `ServerSocketChannel`. In this case, we can be certain that `key.channel()` will return a `ServerSocketChannel`, which is why we can safely cast it without worrying about a `ClassCastException`. The connection is accepted using `serverChannel.accept()`, and as always, we configure it to be non-blocking.

Once the connection is established, we create a `ClientState` object for the client and register the client's channel with the `Selector` for read operations using `SelectionKey.OP_READ`. To make things more manageable, the `ClientState` is attached to the `SelectionKey` as an attachment, allowing us to retrieve it later when needed.

The `ClientState` class plays a crucial role in managing the state of each connected client. It holds the `SocketChannel` for the client, buffers for reading and writing data (`readBuffer` and `responseQueue`), and maintains a `StringBuilder` (`requestBuilder`) to accumulate incoming HTTP request data. This structured per-client state management is essential for handling multiple clients efficiently, ensuring that each connection maintains its own context without interfering with others.

When `key.isReadable()` is true, it means a client has sent data. In this case, we invoke the `handleRead(key)` method.

Let's see what we have in the read handler:

```
    private void handleRead(SelectionKey key) throws IOException {
        SocketChannel channel = (SocketChannel) key.channel();
        ClientState state = (ClientState) key.attachment();

        int bytesRead = channel.read(state.readBuffer); ❶
        if (bytesRead == -1) {
            // Client disconnected
            channel.close();
            return;
        }

        if (bytesRead > 0) {
            state.readBuffer.flip();
            byte[] data = new byte[state.readBuffer.remaining()];
            state.readBuffer.get(data);
            state.requestBuilder.append(new String(data, StandardCharsets.UTF_8
            state.readBuffer.clear();

            // Process complete requests
            processCompleteRequests(state, key); ❷
        }
    }

    private void processCompleteRequests(ClientState state, SelectionKey key) {
        String buffer = state.requestBuilder.toString();
        while (true) {
            HttpRequest request = parseHttpRequest(buffer); ❸
            if (request == null) {
                break; // No complete request found
            }
            state.pendingRequests.offer(request);
            buffer = request.remainingBuffer;
            // Check keep-alive based on HTTP version and headers
            String connection = request.getHeader("Connection");
            if (request.version.equals("HTTP/1.0")) {
                // HTTP/1.0 defaults to close unless keep-alive is explicit
                state.keepAlive = "keep-alive".equalsIgnoreCase(connection);
            } else {
                // HTTP/1.1 defaults to keep-alive unless close is explicit
                state.keepAlive = !"close".equalsIgnoreCase(connection);
            }
        }
        state.requestBuilder = new StringBuilder(buffer);
        if (!state.responseQueue.isEmpty()) {
            key.interestOps(key.interestOps() | SelectionKey.OP_WRITE); ❹
        }
    }
```

The read handler processes incoming data with several important characteristics:

❶ Reading is non-blocking and may return partial data.

❷ We process complete HTTP requests as they arrive, handling partial requests gracefully.

❸ The keep-alive logic now correctly handles both HTTP/1.0 and HTTP/1.1 protocols. HTTP/1.0 defaults to closing connections unless the client explicitly requests keep-alive, while HTTP/1.1 defaults to keeping connections alive unless the client requests closure. This distinction is crucial for compatibility with tools like Apache Bench.

❹ We register interest in writing events only when we have data to send.

The handler reads data from the client's channel and puts it into the `readBuffer`. Once the bytes are received, they are decoded into characters using UTF-8 and appended to the `requestBuilder`. The system processes HTTP requests by looking for the standard HTTP request terminator `("\r\n\r\n")`. When a complete request is detected, the server parses it and processes it asynchronously, ensuring the event loop is never blocked.

When `key.isWritable()` is `true`, it signals that the client's channel is ready to receive data. At this point, the `handleWrite(key)` method takes over, ensuring that any pending responses in the queue are written back to the client efficiently.

Let's look at the code:

```
private void handleWrite(SelectionKey key) throws IOException {
    SocketChannel channel = (SocketChannel) key.channel();
    ClientState state = (ClientState) key.attachment();

    while (!state.responseQueue.isEmpty()) {
        String response = state.responseQueue.peek();
        ByteBuffer buffer
        = ByteBuffer.wrap(response.getBytes(StandardCharsets.UTF_8));

        int written = channel.write(buffer);
        if (buffer.hasRemaining()) {
            // Socket buffer is full, try again later
            break;
        }

        // Response fully written, remove from queue
        state.responseQueue.poll();
    }

    if (state.responseQueue.isEmpty()) {
        // No more data to write, stop watching for write events
        key.interestOps(key.interestOps() & ~SelectionKey.OP_WRITE);

        if (!state.keepAlive && state.pendingRequests.isEmpty()) {
            // Close connection if not keep-alive and no pending requests
```

```
                System.out.println("Closing connection: "
                                   + channel.getRemoteAddress());
                channel.close();
                key.cancel();
            }
        }
    }
```

The handler iterates through the client's `responseQueue`, writing each enqueued HTTP response to the client's channel using non-blocking writes. If the socket's buffer becomes full, the writing process pauses, waiting for the next `OP_WRITE` event before resuming. Once all responses are successfully written and the `responseQueue` is empty, the server removes the `OP_WRITE` interest from the `Selector` for that client, ensuring that no unnecessary write events are triggered when there's nothing left to send. The thread-safe `ConcurrentLinkedQueue` ensures that responses added by async threads don't cause race conditions during this process.

After handling all the selected keys, we process any pending requests that were parsed:

```
    private void processAllPendingRequests(Selector selector) {
        for (SelectionKey key : selector.keys()) {
            if (!key.isValid() || key.attachment() == null)
                continue;
            ClientState state = (ClientState) key.attachment();

            if (state.isProcessing || state.pendingRequests.isEmpty())
                continue;

            state.isProcessing = true; ❶
            while (!state.pendingRequests.isEmpty()) {
                HttpRequest request = state.pendingRequests.poll();
                int requestId = requestCounter.incrementAndGet();
                System.out.println("Request #" + requestId + ": "
                        + request.method + " " + request.path);
                // Simulate async processing
                CompletableFuture.runAsync(() -> { ❷
                    String response;
                    try {
                        // Process request (simulate the work)
                        if (request.path.equals("/slow")) {
                            Thread.sleep(2000); // Simulate slow operation
                            System.out.println("  Slow request processed");
                        } else {
                            System.out.println("  Fast request processed");
                        }
                        response = buildHttpResponse(request, requestId,
                                                     state.keepAlive);
                    } catch (Exception e) {
```

```
                        System.err.println("Error processing request #" + reques
                        + ": " + e.getMessage());
                        response = buildErrorResponse(request, requestId, e);
                    }
                    state.responseQueue.offer(response); ❸

                    // Queue the update for the selector thread
                    pendingUpdates.offer(new PendingUpdate(key)); ❹
                    selector.wakeup();
                });
            }

            state.isProcessing = false;
        }
    }
```

The power of asynchronous processing becomes evident here:

❶ The processing flag ensures we don't start processing the same client's requests multiple times if events trigger in quick succession.

❷ Slow operations are handled asynchronously without blocking the event loop.

❸ Responses are safely added to the concurrent queue from the async thread.

❹ Instead of modifying the `SelectionKey` directly from the async thread (which would cause race conditions), we queue an update that the main selector thread will process. This is a critical pattern for thread safety in NIO applications.

NOTE

A common misconception arises from the Java documentation stating that "Selection keys are safe for use by multiple concurrent threads." This leads many developers to believe they can modify `interestOps` from any thread. In reality, this statement only means that reading key properties won't cause memory corruption. It does not mean that operations like `interestOps()` are atomic or safe for concurrent modification.

Modifying a `SelectionKey`'s `interestOps` while `selector.select()` is executing can lead to:

- Race conditions where `interestOps` changes are lost
- Missed I/O events
- `CancelledKeyException` errors
- Inconsistent behavior that often only appears under high load

Always modify `SelectionKey`'s `interestOps` from the same thread that runs the selector loop. This is why we use the `pendingUpdates` queue pattern; it ensures all modifications happen safely in the selector thread.

Next, we will discuss the helper methods that complete the non-blocking HTTP server implementation.

The `buildHttpResponse` method constructs a properly formatted HTTP response with headers and body:

```
  private String buildHttpResponse(HttpRequest request,
                                   int requestId, boolean keepAlive) {
      StringBuilder response = new StringBuilder();
      // Match the request's HTTP version
      response.append(request.version).append(" 200 OK\r\n");
      // Headers
      response.append("Content-Type: text/plain\r\n");
      response.append("Server: NonBlockingHttpServer/1.0\r\n");
      response.append("Date: ").append(Instant.now()).append("\r\n");
      // Handle connection header properly
      if (request.version.equals("HTTP/1.0") && keepAlive) {
        response.append("Connection: keep-alive\r\n");

        return null;
      } else if (request.version.equals("HTTP/1.1") && !keepAlive) {
        response.append("Connection: close\r\n");
      }
      // For HTTP/1.1 with keep-alive, no Connection header needed (it's defau
      String body = String.format(
          "Request #%d processed\nPath: %s\nMethod: %s\nTime: %s\nThread: %s\n
          requestId,
          request.path,
          request.method,
          Instant.now(),
          Thread.currentThread().getName());
```

```
        response.append("Content-Length: ").append(body.length()).append("\r\n"
        response.append("\r\n"); // Empty line between headers and body
        response.append(body);
        return response.toString();
    }
```

A crucial thing to note here is that the response now echoes the client's HTTP version instead of always returning HTTP/1.1. This ensures compatibility with HTTP/1.0 clients like Apache Bench. The `Connection` header is also handled correctly based on the protocol version and keep-alive status.

The heart of request handling is parsing the raw HTTP data. The `parseHttpRequest` method extracts HTTP requests from the byte stream, handling partial data gracefully:

```
  private HttpRequest parseHttpRequest(String buffer) {
      int requestEndIndex = buffer.indexOf("\r\n\r\n");
      if (requestEndIndex == -1) {
        requestEndIndex = buffer.indexOf("\n\n");
        if (requestEndIndex == -1) {
          return null; // No complete request yet
        }
      }

      String requestText = buffer.substring(0, requestEndIndex);
      String[] lines = requestText.split("\r\n|\n");
      if (lines.length == 0) return null;

      // Parse request line
      String[] requestLineParts = lines[0].split(" ");
      if (requestLineParts.length != 3) return null;
      HttpRequest request = new HttpRequest();
      request.method = requestLineParts[0];
      request.path = requestLineParts[1];
      request.version = requestLineParts[2];

      for (int i = 1; i < lines.length; i++) {
        String line = lines[i];
        int colonPos = line.indexOf(':');
        if (colonPos > 0) {
          String name = line.substring(0, colonPos).trim();
          String value = line.substring(colonPos + 1).trim();
          request.headers.put(name.toLowerCase(), value);
        }
      }

      // Store remaining buffer for next request
      request.remainingBuffer = buffer.substring(
          requestEndIndex
              + (buffer.substring(requestEndIndex)
                      .startsWith("\r\n\r\n") ? 4 : 2));
```

```
        return request;
    }
```

When request processing fails, we need a specialized error response. The `buildErrorResponse` method creates an appropriate HTTP 500 error response:

```
    private String buildErrorResponse(HttpRequest request,
                                      int requestId, Exception error) {
        StringBuilder response = new StringBuilder();
        // Match the request's HTTP version
        response.append(request.version).append(" 500 Internal Server Error\r\n

        // Headers
        response.append("Content-Type: text/plain\r\n");
        response.append("Server: NonBlockingHttpServer/1.0\r\n");
        response.append("Date: ").append(Instant.now()).append("\r\n");
        // Always close connection on error
        response.append("Connection: close\r\n");

        // Body
        String body = String.format(
            "Request #%d failed\nPath: %s\nError: %s\nTime: %s",
            requestId,
            request.path,
            error.getMessage() != null
                ? error.getMessage()
                : error.getClass().getSimpleName(),
            Instant.now());

        response.append("Content-Length: ")
            .append(body.length()).append("\r\n")
            .append("\r\n") // Empty line between headers and body
            .append(body);

        return response.toString();
    }
```

The `HttpRequest` class remains exactly the same as in our previous example, so we won't repeat it here. It contains the HTTP method, path, version, headers map, and the remaining buffer for handling partial requests in our non-blocking server.

If we run the Java program now, even though it operates with a single thread, it can handle multiple connections simultaneously. This is the power of NIO: it can efficiently manage numerous clients without spawning a separate thread for each connection.

To test its scalability, let's create a simple multiclient application that can send thousands of connection requests to this server, pushing it to handle

a large volume of concurrent connections and observing how it performs under load:

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.Socket;
import java.util.ArrayList;
import java.util.List;

public class PipeliningLoadTest {
  public static void main(String[] args) throws Exception {
    int numConnections = 10;
    int requestsPerConnection = 100;
    long startTime = System.currentTimeMillis();
    List<Thread> threads = new ArrayList<>();
    for (int i = 0; i < numConnections; i++) {
      Thread t = Thread.ofVirtual().start(() -> { ❶
        try {
          testPipelining(requestsPerConnection);
        } catch (Exception e) {
          e.printStackTrace();
        }
      });
      threads.add(t);
    }
    for (Thread t : threads) {
      t.join();
    }
    long elapsed = System.currentTimeMillis() - startTime;
    System.out.println("Total time: " + elapsed + "ms");
    System.out.println("Requests per second: " +
        (numConnections * requestsPerConnection * 1000.0 / elapsed));
  }

  private static void testPipelining(int numRequests)
      throws Exception {
    Socket socket = new Socket("localhost", 8080);
    PrintWriter out = new PrintWriter(
        socket.getOutputStream(), true);
    BufferedReader in = new BufferedReader(
        new InputStreamReader(socket.getInputStream()));
    for (int i = 0; i < numRequests; i++) {
      String path = (i % 10 == 0) ? "/slow" : "/fast"; ❷
      out.println("GET " + path + " HTTP/1.1");
      out.println("Host: localhost");
      out.println("Connection: " +
          (i == numRequests - 1 ? "close" : "keep-alive")); ❸
      out.println();
    }
    for (int i = 0; i < numRequests; i++) {
      readResponse(in); ❹
    }
```

```
      socket.close();
    }

    static void readResponse(BufferedReader in)
        throws IOException {
      // Read status line and headers
      String line;
      int contentLength = 0;
      while ((line = in.readLine()) != null) {
        System.out.println(line);
        if (line.startsWith("Content-Length: ")) {
          contentLength = Integer.parseInt(
              line.substring("Content-Length: ".length()));
        }
        if (line.isEmpty()) {
          break; // End of headers
        }
      }
      // Read body
      char[] body = new char[contentLength];
      in.read(body);
      System.out.println(new String(body));
    }
  }
```

This load test demonstrates several important aspects of HTTP pipelining and concurrent connection handling:

1. We use virtual threads to efficiently create 10 concurrent connections. Each virtual thread handles one connection with 100 pipelined requests.
2. Every tenth request is slow (two-second delay) to simulate a realistic workload mix.
3. The last request on each connection uses `"Connection: close"` to properly terminate the keep-alive session.
4. After sending all requests, we read all responses in order—this is the essence of true HTTP/1.1 pipelining.

The load test spins up 10 virtual-thread clients, each opening a single persistent connection to the server and pipelining 100 HTTP/1.1 requests down that socket (1,000 requests total). Every tenth request targets the `/slow` path, pausing for two seconds to simulate heavy work; the other nine hit `/fast` and return immediately. After dispatching all requests, each client reads the responses in order, preserving pipeline semantics. When all threads join, the harness prints the total wall-clock time and calculates the throughput in requests per second. Because only 10 OS sockets are active and the server's selector handles readiness events in one thread, the test stresses connection concurrency and pipelining efficiency rather than raw socket count. This makes it ideal for highlighting NIO's

ability to keep throughput high even when individual requests vary widely in latency.

Event-Driven Architecture

While our simple guessing game scales well and serves its purpose, real-world applications are far more complex. They involve much more than just handling a few guesses from a client. So, how do we structure our code in a way that supports larger, more dynamic systems?

Fortunately, many frameworks have been built on top of the event-driven paradigm, making it easier to efficiently handle concurrency and scalability. Frameworks like Netty and Vert.x are great examples of this approach.

If we were to rewrite our guessing game using Vert.x, the implementation would look something like this:

```
import io.vertx.core.AbstractVerticle;
import io.vertx.core.Promise;
import io.vertx.core.Vertx;
import io.vertx.core.json.JsonObject;
import io.vertx.ext.web.Router;
import io.vertx.ext.web.RoutingContext;
import java.util.concurrent.atomic.AtomicLong;

public class VertxHttpServer extends AbstractVerticle {
  private static final int PORT = 8080;
  private final AtomicLong requestCounter = new AtomicLong(0); ❶
  private long startTime;

  @Override
  public void start(Promise<Void> startPromise) {
    startTime = System.currentTimeMillis();
    Router router = Router.router(vertx); ❷
    router.get("/fast").handler(this::handleFastRequest);
    router.get("/slow").handler(this::handleSlowRequest);
    router.get("/stats").handler(this::handleStats);
    vertx.createHttpServer()
        .requestHandler(router)
        .listen(PORT, "localhost"); ❸
  }

  private void handleFastRequest(RoutingContext ctx) {
    long requestId = requestCounter.incrementAndGet();
    ctx.response()
        .putHeader("content-type", "text/plain")
        .end("Request #" + requestId + ": Fast request processed"); ❹
  }

  private void handleSlowRequest(RoutingContext ctx) {
```

```
    long requestId = requestCounter.incrementAndGet();
    vertx.setTimer(2000, id -> { ❺
      ctx.response()
          .putHeader("content-type", "text/plain")
          .end("Request #" + requestId + ": Slow request processed");
    });
  }

  private void handleStats(RoutingContext ctx) {
    long uptimeMillis = System.currentTimeMillis() - startTime;
    JsonObject stats = new JsonObject()
            .put("totalRequests", requestCounter.get())
            .put("uptimeMillis", uptimeMillis)
            .put("currentThread", Thread.currentThread().getName())
            .put("isEventLoopThread", Vertx.currentContext()
                                .isEventLoopContext()); ❻
    ctx.response()
        .putHeader("content-type", "application/json")
        .end(stats.encodePrettily());
  }

  public static void main(String[] args) {
    Vertx vertx = Vertx.vertx();
    vertx.deployVerticle(new VertxHttpServer()); ❼
  }
}
```

This Vert.x implementation showcases several key principles of event-driven architecture:

❶ We use `AtomicLong` for thread-safe request counting, since multiple event loop threads might increment it concurrently.

❷ The `Router` handles HTTP routing declaratively; each route is mapped to a handler method.

❸ The server starts asynchronously on the event loop without blocking the main thread.

❹ Fast requests complete immediately on the event loop thread, demonstrating non-blocking I/O.

❺ The crucial difference: `vertx.setTimer()` schedules the response after two seconds *without* blocking the event loop thread. This is the key to maintaining high concurrency.

❻ The `stats` endpoint reveals which thread is handling the request, helping us verify that all requests run on event loop threads.

❼ Deploying a `verticle` starts the event-driven server with Vert.x managing the event loops automatically.

To run this Vert.x server, you'll need to add the following dependencies to your Maven project. Create a Maven project, then add these dependencies and run it:

```
<dependency>
    <groupId>io.vertx</groupId>
    <artifactId>vertx-core</artifactId>
    <version>5.0.0</version>
</dependency>
<dependency>
    <groupId>io.vertx</groupId>
    <artifactId>vertx-web</artifactId>
    <version>5.0.0</version>
</dependency>
```

This gives us HTTP endpoints where we can test with curl:

```
curl -X GET http://localhost:8080/fast
curl -X GET http://localhost:8080/slow
curl -X GET http://localhost:8080/stats
```

Let's focus on the high-level concept rather than diving into the intricate details of how Vert.x is implemented under the hood. At its core, Vert.x is built on a multicore reactor pattern, leveraging an event-loop concurrency model. It uses a lightweight set of event loop threads that continuously listen for I/O events via non-blocking operations such as NIO. Whenever an event occurs, such as an incoming request, a short, non-blocking callback is executed within the event loop, ensuring responsiveness.

However, not all tasks can be handled in an event loop. Some operations are inherently blocking or long-running, such as database queries or filesystem access. Vert.x intelligently offloads them to a separate worker thread pool to prevent these tasks from slowing down the event loop. This design ensures that the event loop remains free to handle new requests without unnecessary delays.

One of the key strengths of Vert.x is its event bus, which allows different components of an application to communicate asynchronously. This approach keeps the system loosely coupled while ensuring seamless coordination between different parts of the application (Figure 6-1).

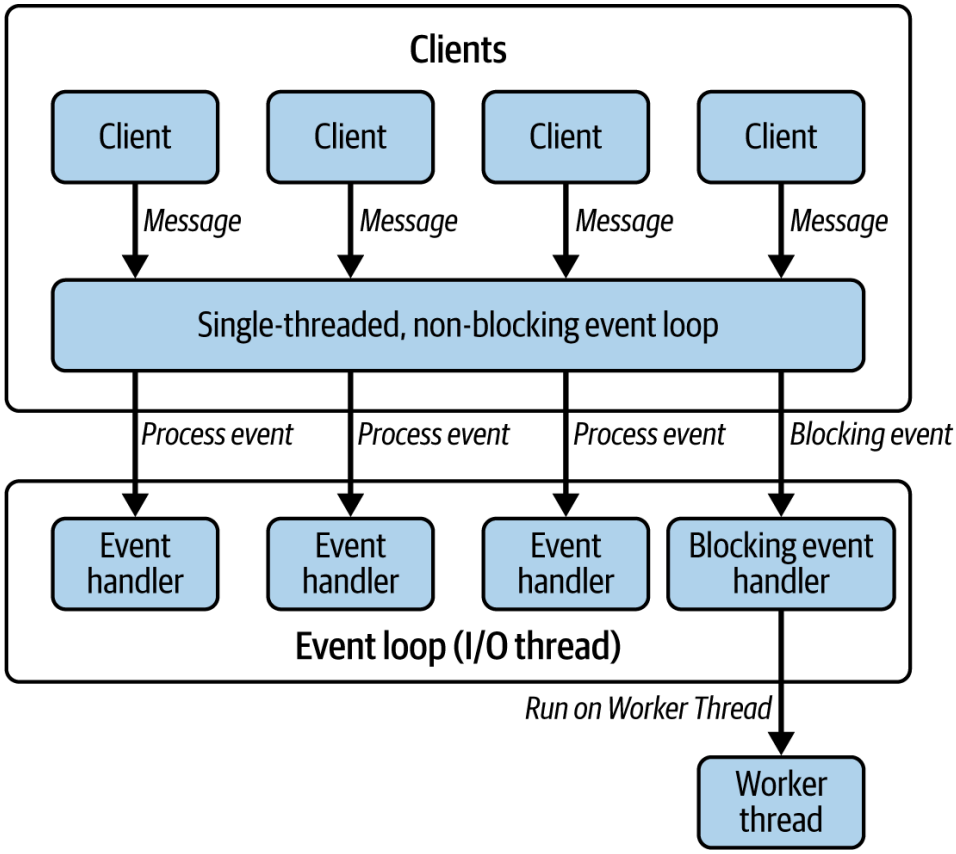

Figure 6-1. Overview of event loop of event-driven architecture in Vert.x

In Figure 6-1, we get a glimpse of how a framework like Vert.x operates with the event loop at its core. This framework is responsible for managing client connections, handling outbound requests, and writing responses—all in a non-blocking manner. Typically, a reactive framework sits on top of this, providing a higher-level API that simplifies network operations. We'll explore these in more detail soon. Instead of dealing with raw, non-blocking I/O directly, developers work with intuitive abstractions, such as HTTP requests, responses, and Kafka messages, making the development experience far more streamlined.

Our application code sits at the highest layer. Our code doesn't interact directly with the event loop; instead, it works through event handlers. For example, in our earlier code snippet, we created handlers like `handleFastRequest` and `handleSlowRequest` to process specific HTTP routes.

So far, everything seems great. However, here's the critical piece: the event handlers in your application are executed using the event loop thread—essentially an I/O thread. If our code blocks this thread, no other concurrent events can be processed. The result is a complete breakdown in responsiveness and concurrency that would be catastrophic for a system designed to be highly reactive.

Notice how in our Vert.x example, the slow request handler uses `vertx.setTimer()` instead of `Thread.sleep()`. This is crucial—the timer schedules a callback to run after two seconds without blocking the

event loop thread. The thread is immediately free to handle other requests while waiting.

The solution is simple: your code must be non-blocking. It should never, under any circumstance, block the I/O threads. If it does, the entire event loop comes to a standstill, dragging down the responsiveness of your application as well.

There are workarounds, of course. You could offload blocking operations to a separate worker thread pool—something Vert.x itself does to some extent. But this should never be the norm. In fact, relying too much on worker threads defeats the purpose of a reactive system. Why? Because every time you switch execution from an I/O thread to a worker thread and back, you introduce context switches. These add overhead, slowing down response times and chipping away at the efficiency gains you set out to achieve in the first place.

So, how do you ensure your application code remains non-blocking? It's one thing to say, "write non-blocking code," but as we've seen, that's easier said than done. The reality is that writing truly non-blocking code isn't always straightforward; it requires a shift in mindset.

That's where we can really focus on reactive frameworks. They provide asynchronous APIs specifically to help us avoid blocking operations while still handling complex workflows efficiently. Instead of waiting for a task to complete, we register callbacks, use promises, or leverage reactive streams to orchestrate execution without ever stalling the event loop.

But what exactly are these asynchronous APIs, and how do they work? Let's explore.

Asynchronous APIs

Most of us are familiar with synchronous APIs, but let's take a moment to revisit the concept. Consider the following simple Java method:

```
public class AiService {
    public String chat(String message) {
        return "Echo: " + message.toUpperCase();
    }
}
```

This method takes a string as input, processes it, and returns a response. We can invoke it like this:

```
String result = aiService.chat("What is the meaning of life?");
```

In this case, the caller thread executes the chat method and blocks execution until it receives a response. This is a classic example of a *synchronous API*—the caller has to wait.

Now, what if we want to make this API asynchronous? That is, instead of waiting for the result, we let the method return immediately, and once the response is ready, a callback function handles it. Let's modify our method accordingly:

```
public void chat(String message, Consumer<String> consumer) {
    Thread.startVirtualThread(() -> {
        try {
            String response = "Echo: " + message.toUpperCase();
            consumer.accept(response);
        } catch (Exception e) {
            consumer.accept("Error during chat: " + e.getMessage());
        }
    });
}
```

This version of the method now takes two parameters:

1. The message itself
2. A callback function (of type `Consumer<String>`) that will be invoked when the response is ready

Now, let's see how we can use this asynchronous API:

```
aiService.chat("Hello, how are you?", response -> {
    System.out.println("Response 1: " + response);
});
aiService.chat("What is your name?", response -> {
    System.out.println("Response 2: " + response);
});
```

Here's what happens: the first `chat` call is made, but unlike before, the main thread does *not* wait for the result. Instead, it immediately moves to execute the second `chat` call. Meanwhile, the responses will be processed asynchronously when they become available. This is the essence of asynchronous programming—freeing up the main thread to perform other tasks while waiting for results.

However, the callback has some inherent issues. It doesn't integrate well. What if we want to pass the result from the first callback to another chat method? Let's try to do so:

```
aiService.chat("What is the meaning of life?", response -> {
  aiService.chat(response, response2 -> {
```

```
        aiService.chat(response2, response3 -> {
            aiService.chat(response3, response4 -> {
                System.out.println(response4);
            });
        });
    });
});
```

As we can see here, using callbacks for asynchronous execution can quickly become unmanageable. The more callbacks we introduce, the harder it becomes to read and maintain our code, leading to what's often called callback hell.

So, what's the alternative?

Java provides a much cleaner solution: `CompletableFuture`. It allows us to write asynchronous code that remains readable, structured, and composable. Let's modify our `chat` method to return a `CompletableFuture<String>` instead:

```
public CompletableFuture<String> chat(String message) {
    return CompletableFuture.supplyAsync(() -> {
        try {
            return "Echo: " + message.toUpperCase();
        } catch (Exception e) {
            return "Error during chat: " + e.getMessage();
        }
    });
}
```

Now, instead of relying on callbacks, we can chain multiple asynchronous calls together in a more structured way:

```
aiService.chat("What is the meaning of life?")
        .thenCompose(aiService::chat)
        .thenCompose(aiService::chat)
        .thenCompose(aiService::chat)
        .thenAccept(System.out::println);
```

This makes composing asynchronous methods much easier than dealing with nested callbacks. We haven't touched on error handling yet, but `CompletableFuture` provides a rich API for gracefully managing failures.

So, does this mean we've solved the problem of blocking I/O threads? Well, while `CompletableFuture` does a great job of handling single asynchronous results, there's something we haven't considered yet: streams of data. `CompletableFuture` works beautifully for methods

that return a single value, but what happens when we need to deal with sequences of results—a continuous flow of data rather than a one-time response?

That's where things get a little more interesting. In fact, that's where the reactive framework shines best.

Reactive Programming in Java

Reactive programming in Java is a paradigm centered around asynchronous, non-blocking, and event-driven applications. A core concept is the *reactive stream*, a standard (defined by the Reactive Streams Specification) for managing asynchronous data flows with *backpressure*.

Let's discuss what a reactive stream is and then talk about backpressure, one by one.

Understanding Reactive Streams

In reactive programming, we organize our code around streams, creating chains of transformations known as pipelines. In this paradigm, everything can be viewed as a stream of events flowing from producers to consumers through a series of transformations.

Events can be anything: user clicks, sensor readings, incoming messages, etc. They travel from an upstream source to a downstream Subscriber, passing through operators that transform or filter the data (Figure 6-2).

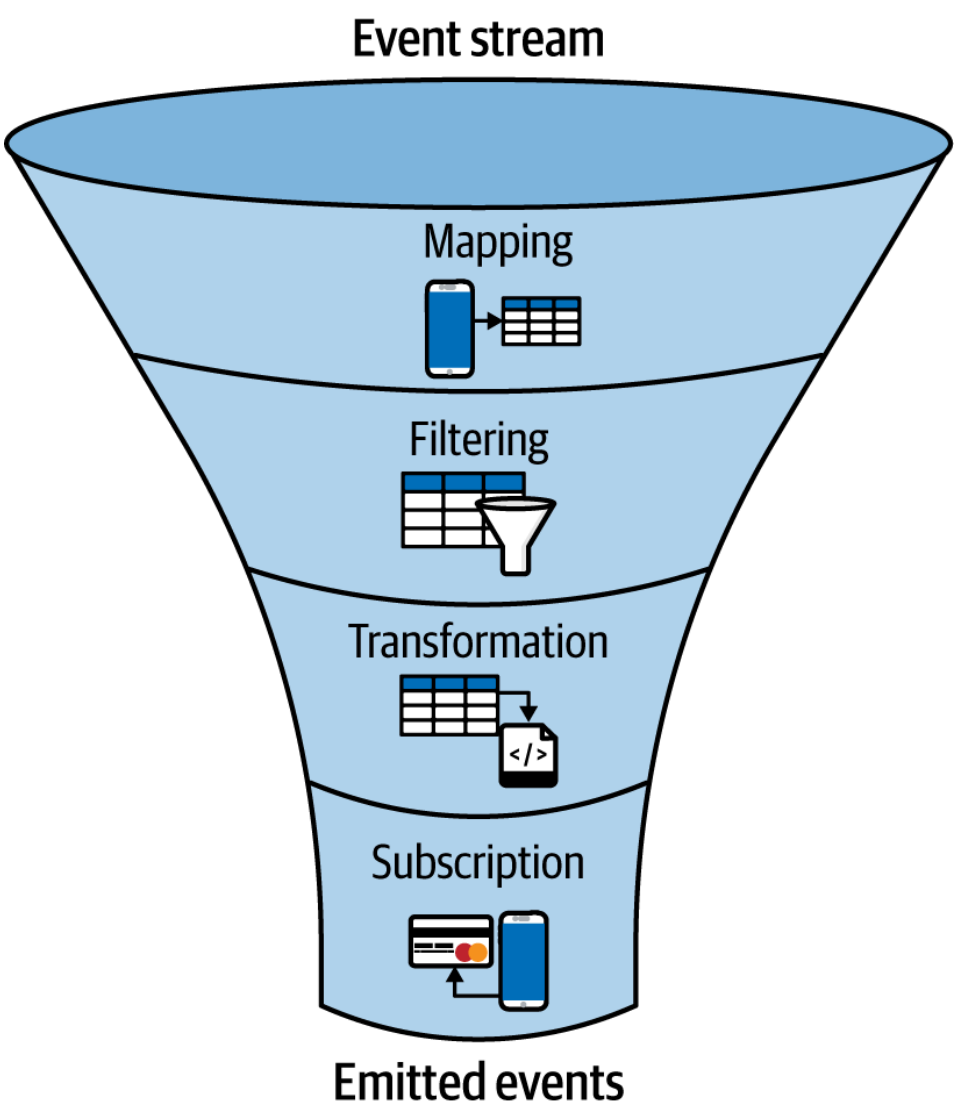

Figure 6-2. Reactive programming data flow

Each operator observes its upstream and produces a new stream. However, streams are lazy by default—they don't start processing until there's a Subscriber. We don't know *when* an event will arrive (asynchronous), so we set up "observers" to react when it does.

In reactive programming, we work with four fundamental components:

Publisher

The source that emits data items asynchronously. Think of it as a data producer that can emit zero or more items over time.

Subscriber

The consumer that receives and processes emitted items. Subscribers express interest in receiving data and define how it should be handled.

Subscription

The link between a Publisher and a Subscriber, managing the flow of data and enabling backpressure control.

Processor

A component that acts as both Subscriber and Publisher, transforming data as it flows through the stream.

A reactive stream can emit three types of signals:

Data items

The actual values flowing through the stream

Error signal

Indicates that an unrecoverable error occurred, terminating the stream

Completion signal

Indicates successful stream completion with no more items to emit

There are several reactive programming libraries in Java. The most notable ones include:

Project Reactor

Provides the core types:

- `Flux<T>` (0 to N items)
- `Mono<T>` (0 to 1 items)

RxJava

A reactive extension for Java. It provides the types:

- `Observable<T>` (0 to N items)
- `Single<T>` (exactly 1 item)
- `Maybe<T>` (0 to 1 items)

Let's explore reactive programming with Project Reactor through a series of examples:

```
public class ReactiveExample {
    public static void main(String[] args) {
        // Create a Flux emitting integers 1 to 5
        Flux<Integer> numbers = Flux.just(1, 2, 3, 4, 5); ❶
        // Process the stream: filter even numbers and convert to strings
        numbers
            .filter(n -> n % 2 == 0) ❷ // Keep even numbers
            .map(n -> "Value: " + n) ❸ // Transform to strings
            .subscribe(
                System.out::println, ❹ // onNext: print each value
                error -> System.err.println("Error: " + error), ❺ // onErro
                () -> System.out.println("Done!") ❻ // onComplete
            );
    }
}
```

This code demonstrates the basic reactive stream pipeline:

❶ Creates a cold stream with five integer values. The stream won't emit any values until a Subscriber subscribes.

❷ The `filter` operator creates a new stream containing only elements that match the predicate, in this case, even numbers.

❸ The `map` operator transforms each element, converting integers to formatted strings.

❹ The `subscribe` method triggers the pipeline execution and defines how to handle each emitted value.

❺ The error-handling callback executes if any operator in the pipeline throws an exception.

❻ The completion callback executes when the stream completes all emissions.

The output would be:

```
Value: 2
Value: 4
Done!
```

Now that we have a basic idea of what it looks like, let's explore reactive programming through a practical example: building a cryptocurrency price-monitoring system.

Imagine we're building a system that monitors cryptocurrency prices from multiple exchanges in real time. This scenario perfectly illustrates the strengths of reactive programming, which include handling multiple asynchronous data sources, transforming data streams, and reacting to specific conditions.

First, let's define our data model:

```
public record PriceData(String exchange, String symbol, double price,
                        Instant timestamp) {}

public record PriceAlert(String symbol, String message, AlertType type) {}

enum AlertType {THRESHOLD_CROSSED, RAPID_CHANGE, ANOMALY}
```

Now, let's create a simple reactive price-monitoring system:

```
import reactor.core.publisher.Flux;
import java.time.Duration;
import java.time.Instant;

public class SimplePriceMonitor {
    public static void main(String[] args) throws InterruptedException {
        // Create a stream of price updates ❶
```

```
            Flux<PriceData> priceStream = Flux.interval(Duration.ofSeconds(1))
                .map(i -> new PriceData(
                    "Binance",
                    "BTC/USD",
                    50000 + (Math.random() - 0.5) * 1000, ❷
                    Instant.now()
                ));

            // Process the stream ❸
            priceStream
                .filter(price -> price.price() > 50200) ❹
                .map(price -> String.format("BTC price $%.2f exceeds threshold!
                                            price.price())) ❺
                .subscribe(
                    alert -> System.out.println(alert), ❻
                    error -> System.err.println("Error: " + error),
                    () -> System.out.println("Monitoring complete")
                );

            // Keep the main thread alive
            Thread.sleep(10000);
        }
    }
```

Let's examine what's happening in this code:

❶ Creates a cold stream that emits a value every second. The `interval` operator uses a daemon thread, so we need to keep the main thread alive. The stream doesn't start emitting until a Subscriber subscribes to it.

❷ Generates simulated price data with random fluctuations around a $50,000 base price.

❸ Begins the stream-processing pipeline. Each operator creates a new stream that observes the previous one.

❹ Filters the stream to only include prices above $50,200, demonstrating selective processing.

❺ Transforms the price data into alert messages, showing how data can be reshaped as it flows.

❻ Subscribes to the stream, triggering the entire pipeline to start processing.

Real-world applications require more sophisticated stream processing. Let's build a comprehensive price monitoring system that handles multiple exchanges and symbols:

```
public class CryptoPriceMonitor {
    private static final List<String> EXCHANGES =
        List.of("Binance", "Coinbase", "Kraken");
    private static final List<String> SYMBOLS =
```

```
        List.of("BTC/USD", "ETH/USD", "SOL/USD");

    // Sinks for broadcasting alerts to multiple Subscribers ❶
    private static final Sinks.Many<PriceAlert> alertSink =
        Sinks.many().multicast().onBackpressureBuffer();

    public static void main(String[] args) throws InterruptedException {
        // Create merged stream from multiple exchanges ❷
        Flux<PriceData> priceStream = Flux.merge(
            EXCHANGES.stream()
                .map(CryptoPriceMonitor::createExchangeFeed)
                .toList()
        );

        // Group prices by symbol for parallel processing ❸
        priceStream
            .groupBy(PriceData::symbol)
            .subscribe(symbolFlux -> {
                String symbol = symbolFlux.key();

                // Calculate 5-second moving average ❹
                symbolFlux
                    .window(Duration.ofSeconds(5))
                    .flatMap(window -> calculateMovingAverage(window, symbol
                    .subscribe(avg -> System.out.printf(
                        " %s Moving Avg: $%.2f%n", symbol, avg));

                // Detect rapid price changes ❺
                symbolFlux
                  .buffer(2, 1)
                  .filter(buffer -> buffer.size() == 2)
                  .map(buffer -> detectRapidChange(buffer.get(0), buffer.get
                  .filter(Optional::isPresent)
                  .map(Optional::get)
                  .subscribe(alertSink::tryEmitNext);
                });

        // Subscribe to alerts ❻
        alertSink.asFlux()
            .subscribe(alert -> System.out.printf(" [%s] %s: %s%n",
                alert.type(), alert.symbol(), alert.message()));

        Thread.sleep(30000);
    }

    private static Flux<PriceData> createExchangeFeed(String exchange) {
    return Flux.interval(Duration.ofMillis(
            100 + (int) (Math.random() * 400))) ❼
            .map(i -> {
                String symbol = SYMBOLS.get(
                    (int) (Math.random() * SYMBOLS.size()));
                double basePrice = getBasePrice(symbol);
                double variation = (Math.random() - 0.5) * 0.01;
                double price = basePrice * (1 + variation);
```

```
                    return new PriceData(exchange, symbol, price,
                                         Instant.now());
                })
                .doOnNext(price -> System.out.printf(
                    "%s [%s]: $%.2f%n", ❽
                    price.exchange(), price.symbol(), price.price()));
    }
```

This advanced example demonstrates several key reactive patterns:

❶ Sinks provide a bridge between imperative and reactive code, allowing manual emission of items into a stream.

❷ `merge` combines multiple streams into a single stream, interleaving items as they arrive.

❸ `groupBy` partitions a stream into multiple substreams based on a key function, enabling parallel processing of different symbols.

❹ `window` creates time-based chunks of data, perfect for calculating metrics over sliding time windows.

❺ `buffer` collects items into lists, with sliding windows created by the `stride` parameter.

❻ Hot streams (created by sinks) allow multiple Subscribers to receive the same events.

❼ Irregular intervals simulate realistic, nonuniform data arrival from different exchanges.

❽ Side effects with `doOnNext` allow logging without affecting the stream's data flow.

There are helper methods that complete this example.

The `calculateMovingAverage` method computes the average price over a sliding window, which is essential for identifying price trends and smoothing out short-term fluctuations:

```
    private static Mono<Double> calculateMovingAverage(Flux<PriceData> window,
                                                       String symbol) {
        return window
            .map(PriceData::price)
            .reduce(new double[]{0, 0}, (acc, price) -> {
              acc[0] += price;  // sum
              acc[1] += 1;      // count
              return acc;
            })
            .map(acc -> acc[1] > 0 ? acc[0] / acc[1] : 0.0)
            .filter(avg -> avg > 0);
    }
```

While moving averages help identify trends, we also need to detect sudden price movements. The `detectRapidChange` method compares consecutive price points to alert on significant changes:

```
  private static Optional<PriceAlert> detectRapidChange(PriceData prev,
                                                        PriceData current) {
      if (!prev.symbol().equals(current.symbol())) {
        return Optional.empty();
      }

      double changePercent = Math.abs((current.price() - prev.price())
          / prev.price()) * 100;
      if (changePercent > 0.5) { // 0.5% change threshold
        return Optional.of(new PriceAlert(
            current.symbol(),
            String.format("Rapid %.2f%% change: $%.2f → $%.2f",
                changePercent, prev.price(), current.price()),
            AlertType.RAPID_CHANGE
        ));
      }
      return Optional.empty();
    }
```

The `getBasePrice` method provides these baseline values for our supported trading pairs:

```
    private static double getBasePrice(String symbol) {
      return switch (symbol) {
        case "BTC/USD" -> 50000;
        case "ETH/USD" -> 3000;
        case "SOL/USD" -> 100;
        default -> 1000;
      };
    }
```

This is a high-level introduction to reactive streams. This book isn't intended to cover every detail but rather to provide you with a taste of what reactive programming entails. If you wish to learn more, I recommend exploring dedicated books on the subject.

NOTE

Reactive streams are not the same as Java Streams.

- `java.util.stream.Stream` (Java Streams API) is used to process collections synchronously and in-memory.
- Reactive streams handle asynchronous, event-driven, and non-blocking data processing with support for backpressure.

Backpressure

In reactive programming, backpressure is an integral mechanism that enables a downstream consumer (Subscriber) to signal an upstream producer (Publisher) when it is unable to process data. Essentially, it will allow the consumer to say, “Slow down! I can’t process this fast enough.”

Let’s examine different backpressure strategies using a high-frequency trading scenario:

```
public class BackpressureDemo {
    public static void main(String[] args) throws InterruptedException {
        // Simulate ultra-high-frequency price feed ❶
        Flux<PriceData> extremeFeed = Flux.interval(Duration.ofNanos(100_000
            .map(i -> new PriceData(
                "HFT-Exchange",
                "BTC/USD",
                50000 + ThreadLocalRandom.current().nextDouble(-100, 100),
                Instant.now()
            ))
            .share(); // Hot stream shared among subscribers

        // Strategy 1: Sampling - Take periodic snapshots ❷
        System.out.println("SAMPLING Strategy:");
        extremeFeed
            .sample(Duration.ofMillis(100))
            .take(10)
            .subscribe(price -> System.out.printf(
                "[SAMPLED] Price: $%.2f at %s%n",
                price.price(), price.timestamp()));

        Thread.sleep(1500);

        // Strategy 2: Drop - Discard when overwhelmed ❸
        System.out.println("\nDROP Strategy:");
        AtomicInteger dropped = new AtomicInteger(0);
        extremeFeed
            .onBackpressureDrop(price -> {
                if (dropped.incrementAndGet() % 1000 == 0) {
                    System.out.printf("[DROPPED] %d updates dropped%n",
                                      dropped.get());
                }
            })
            .publishOn(Schedulers.boundedElastic()) ❹
            .take(Duration.ofSeconds(1))
            .subscribe(price -> {
                simulateWork(10); // Simulate slow processing
                System.out.printf("[PROCESSED] Price: $%.2f%n", price.price
            });

        Thread.sleep(1500);

        // Strategy 3: Latest - Keep only most recent value ❺
```

```
            System.out.println("\nLATEST Strategy:");
            extremeFeed
                .onBackpressureLatest()
                .publishOn(Schedulers.boundedElastic())
                .subscribe(price -> {
                    simulateWork(100);  // Very slow processing
                    System.out.printf("[LATEST] Price: $%.2f%n", price.price())
                });

            Thread.sleep(2000);
        }

      private static void simulateWork(int millis) {
        try {
          Thread.sleep(millis);
        } catch (InterruptedException e) {
          Thread.currentThread().interrupt();
          throw new RuntimeException(e);
        }
      }
    }
```

Let's examine each backpressure strategy:

❶ Creates a stream emitting 10,000 items per second, simulating high-frequency trading data.

❷ Sampling takes periodic snapshots, which is ideal when you need regular updates but can't process every item.

❸ `Drop` discards items when the consumer can't keep up; this is suitable when it's acceptable to miss some data.

❹ `publishOn` moves processing to a different thread pool, separating production from consumption.

❺ `Latest` keeps only the most recent unprocessed item, perfect for scenarios where only the current state matters.

Reactor provides different strategies for handling backpressure:

`onBackpressureBuffer()`

Buffers all items until the consumer catches up. Use when no data can be lost, but memory is available.

`onBackpressureBuffer(maxSize)`

Buffers with a limit, failing if exceeded. Provides safety against memory exhaustion.

`onBackpressureDrop()`

Silently drops excess items. Use for real-time data where the latest values matter more than completeness.

`onBackpressureLatest()`

Keeps only the most recent item. Ideal for status updates where intermediate values become obsolete.

onBackpressureError()

Fails fast when backpressure occurs. Use when backpressure indicates a system design flaw.

This example illustrates how reactive streams function in a real-world scenario, connecting the concepts of Publishers, operators, Subscribers, and backpressure.

Now, how would our price-monitoring system look if implemented with virtual threads?

Let's compare both approaches using our price-monitoring example:

```
package ca.bazlur.mcj.chap6.virtualthreads;
import ca.bazlur.mcj.chap6.reactive.AlertType;
import ca.bazlur.mcj.chap6.reactive.PriceAlert;
import ca.bazlur.mcj.chap6.reactive.PriceData;
import java.time.Instant;
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicReference;

public class PriceMonitorWithVirtualThreads {
 private static final List<String> SYMBOLS =
    List.of("BTC/USD", "ETH/USD", "SOL/USD");
 private static final List<String> EXCHANGES =
    List.of("Binance", "Coinbase", "Kraken");
 private static final Map<String, AtomicReference<PriceData>> latestPrices =
     new ConcurrentHashMap<>();
 private static final BlockingQueue<PriceAlert> alertQueue =
     new LinkedBlockingQueue<>();

 public static void main(String[] args) throws InterruptedException {
   try (var executor = Executors.newVirtualThreadPerTaskExecutor()) { ❶
     // Start price feeds from each exchange
     for (String exchange : EXCHANGES) {
       executor.submit(() -> generatePriceFeed(exchange));
     }
     // Start processors
     executor.submit(PriceMonitorWithVirtualThreads::processAlerts);
     executor.submit(PriceMonitorWithVirtualThreads::monitorThresholds);
     Thread.sleep(30000);
   }
 }

 private static void generatePriceFeed(String exchange) {
   var random = ThreadLocalRandom.current();
   while (!Thread.currentThread().isInterrupted()) {
     try {
       Thread.sleep(100 + random.nextInt(400)); ❷
```

```
        String symbol = SYMBOLS.get(random.nextInt(SYMBOLS.size()));
        double basePrice = getBasePrice(symbol);
        double variation = (random.nextDouble() - 0.5) * 0.01;
        double price = basePrice * (1 + variation);
        PriceData priceData = new PriceData(
          exchange,
          symbol,
          price,
          Instant.now()
        );

        System.out.printf(" %s [%s]: $%.2f%n", exchange, symbol, price);
        // Process this price in a new virtual thread
        Thread.startVirtualThread(() -> processPrice(priceData)); ❸
      } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        break;
      }
    }
  }

  private static void processPrice(PriceData currentPrice) {
    PriceData previousPrice = latestPrices
        .computeIfAbsent(currentPrice.symbol(), k -> new AtomicReference<>()
        .getAndSet(currentPrice); ❹
    if (previousPrice != null) {
      detectRapidChange(previousPrice, currentPrice);
    }
    calculateSimpleMovingAverage(currentPrice.symbol(), currentPrice);
  }

  private static void detectRapidChange(PriceData prev, PriceData current) {
    if (!prev.symbol().equals(current.symbol())) return;
    double changePercent = Math.abs((current.price() - prev.price())
                                    / prev.price()) * 100;
    if (changePercent > 0.5) { // 0.5% change threshold
      PriceAlert alert = new PriceAlert(
          current.symbol(),
          String.format("Rapid %.2f%% change: $%.2f → $%.2f",
              changePercent, prev.price(), current.price()),
          AlertType.RAPID_CHANGE
      );
      alertQueue.offer(alert); ❺
    }
  }

  private static void monitorThresholds() {
    Map<String, Double> thresholds = Map.of(
        "BTC/USD", 51000.0,
        "ETH/USD", 3100.0,
        "SOL/USD", 105.0
    );
    while (!Thread.currentThread().isInterrupted()) {
      try {
```

```
            Thread.sleep(2000); //
            thresholds.forEach((symbol, threshold) -> {
              AtomicReference<PriceData> ref = latestPrices.get(symbol);
              if (ref != null) {
                PriceData latest = ref.get();
                if (latest != null && latest.price() > threshold) {
                  alertQueue.offer(new PriceAlert(
                      symbol,
                      String.format("Price $%.2f exceeded threshold $%.2f",
                          latest.price(), threshold),
                      AlertType.THRESHOLD_CROSSED
                  ));
                }
              }
            });
          } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            break;
          }
        }
      }

      private static final Map<String, LinkedList<Double>> priceWindows =
          new ConcurrentHashMap<>();

      private static void calculateSimpleMovingAverage(String symbol, PriceData p
        var window = priceWindows.computeIfAbsent(symbol, k -> new LinkedList<>(
        synchronized (window) { //
          window.add(price.price());
          if (window.size() > 10) {
            window.removeFirst();
          }
          if (window.size() >= 5) {
            double avg = window.stream()
                .mapToDouble(Double::doubleValue)
                .average()
                .orElse(0.0);
            System.out.printf("%s Moving Avg: $%.2f%n", symbol, avg);
          }
        }
      }

      private static void processAlerts() {
        while (!Thread.currentThread().isInterrupted()) {
          try {
            PriceAlert alert = alertQueue.poll(100, TimeUnit.MILLISECONDS); //
            if (alert != null) {
              System.out.printf(" [%s] %s: %s%n",
                  alert.type(), alert.symbol(), alert.message());
            }
          } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            break;
          }
        }
```

```
    }

    private static double getBasePrice(String symbol) {
      return switch (symbol) {
        case "BTC/USD" -> 50000;
        case "ETH/USD" -> 3000;
        case "SOL/USD" -> 100;
        default -> 1000;
      };
    }
}
```

Key differences in the virtual thread approach:

1. Virtual threads are created implicitly through the executor, making thread creation lightweight and resource-efficient.
2. Explicit loops with sleep calls replace reactive intervals, requiring manual timing control.
3. Each price update spawns a new virtual thread for processing, achieving concurrency through thread creation rather than stream operators.
4. Shared mutable state with atomic references requires careful synchronization, unlike reactive's immutable transformations.
5. Manual synchronization is needed for shared data structures, whereas reactive streams handle this through operators.

Comparison and trade-offs

Looking at both implementations side by side reveals fundamental differences in each approach. The reactive version utilizes a declarative pipeline where data flows through transformations. For instance, `window(Duration.ofSeconds(5))` creates time-based chunks, `onBackpressureDrop()` elegantly handles overflow, and `groupBy()` enables parallel processing without explicit thread management.

In contrast, the virtual threads version uses imperative code with manual state management—maintaining a `LinkedList` with synchronized blocks for moving averages, using `BlockingQueue` for backpressure, and explicitly creating threads with `Thread.startVirtualThread()`. While reactive code reads like a description of data transformations, virtual thread code reads like a sequence of instructions, making it more familiar to developers used to traditional Java programming but requiring more careful attention to synchronization and state management.

Benefits and Downsides of Reactive Programming

Now that we have a high-level understanding of how reactive systems operate, leveraging non-blocking I/O, adopting an event-driven approach, and utilizing stream-based processing, let's explore the advantages and challenges that accompany this paradigm.

Benefits of reactive programming

First and foremost, reactive programming relies on non-blocking operations and asynchronous execution. This allows applications to handle a large number of concurrent requests while consuming minimal system resources. For I/O-bound workloads, this translates into significantly improved scalability and performance.

Second, reactive programming provides a robust toolkit for writing non-blocking and asynchronous code. It encourages a functional and declarative coding style, which many developers find more intuitive and expressive than traditional imperative programming. When used effectively, reactive code can be more readable and maintainable because it composes data streams and handles events in a structured way.

Moreover, reactive programming seamlessly integrates with modern cloud-native applications, making it particularly well-suited for microservices architectures. It enables efficient data flow and communication between distributed components, reducing bottlenecks and improving responsiveness.

Downsides of reactive programming

Despite its many advantages, reactive programming also presents its own challenges.

First, it introduces a fundamentally different programming paradigm. Developers accustomed to imperative programming may find the learning curve steep. Adapting to reactive patterns, such as composing streams and managing backpressure, requires a shift in mindset and a familiarity with new concepts and techniques.

Debugging reactive code can also be notoriously difficult. The asynchronous nature of execution, coupled with event-driven data flow, makes it challenging to trace errors and understand execution paths. The logical flow we see in the code is completely different from the actual execution, making both mental debugging and IDE debugging challenging.

Consider the following example of reactive code that throws an exception:

```
import reactor.core.publisher.Flux;

public class ReactiveErrorExample {
    public static void main(String[] args) {
        Flux.just(1, 2, 3, 0, 5)
                .map(number -> 10 / number)
                .subscribe(
                        System.out::println,
                        Throwable::printStackTrace,
                        () -> System.out.println("Done!")
                );
    }
}
```

Running this code results in an exception:

```
java.lang.ArithmeticException: / by zero
    at.ReactiveErrorExample.lambda$main$0(ReactiveErrorExample.java:8)
    at reactor.core.publisher.FluxMapFuseable$MapFuseableSubscriber.onNext(
    pFuseable.java:113)
    at reactor.core.publisher.FluxArray$ArraySubscription.fastPath(FluxArray
    :171)
    at reactor.core.publisher.FluxArray$ArraySubscription.request(FluxArray
    96)
    at reactor.core.publisher.FluxMapFuseable$MapFuseableSubscriber.request
    apFuseable.java:171)
    at reactor.core.publisher.LambdaSubscriber.onSubscribe(LambdaSubscriber
    119)
    at reactor.core.publisher.FluxMapFuseable$MapFuseableSubscriber.onSubsc
    luxMapFuseable.java:96)
    at reactor.core.publisher.FluxArray.subscribe(FluxArray.java:53)
    at reactor.core.publisher.FluxArray.subscribe(FluxArray.java:59)
    at reactor.core.publisher.Flux.subscribe(Flux.java:8836)
    at reactor.core.publisher.Flux.subscribeWith(Flux.java:8957)
    at reactor.core.publisher.Flux.subscribe(Flux.java:8801)
    at reactor.core.publisher.Flux.subscribe(Flux.java:8725)
    at ReactiveErrorExample.main(ReactiveErrorExample.java:9)
```

While the first line correctly identifies `ReactiveErrorExample.java:8` as the source of the issue, the rest of the stack trace consists of internal calls within the Reactor library. The `threaddump` looks like complete gibberish. This makes debugging more complex compared to traditional, synchronous code, where stack traces are usually straightforward.

Another concern is maintainability. While reactive code can be elegant and concise, it may also be harder to understand and modify. The declarative nature of reactive programming, combined with a rich set of operators, often makes it difficult for developers to predict how the code will behave. This can lead to unintentional side effects when making changes.

Finally, while reactive programming excels at managing I/O-bound workloads, it may not be the ideal choice for CPU-bound tasks, much like virtual threads. Traditional multithreading approaches might offer a more intuitive and manageable solution for certain types of applications, but that's not what we are discussing here.

In Closing

Having explored both traditional concurrency models and their shared focus on handling I/O operations, where does that leave us?

This is where I'll share my perspective. Over the years, I've gained experience, and while I have certain biases, I also appreciate the various use cases for different approaches. Every technology has its strengths and trade-offs. Virtual threads offer simplicity and numerous advantages, but one could argue that they lack a straightforward, built-in way to implement backpressure, something that reactive frameworks handle natively. Of course, we can build backpressure mechanisms ourselves, but that responsibility falls squarely on the developer's shoulders.

That said, both paradigms share a common goal: to efficiently manage I/O-bound operations. If I had to choose between them, I would lean toward virtual threads. Reactive programming maintains its relevance, particularly in scenarios that require fine-grained backpressure control, complex data transformations, or event-driven microservices.

Virtual threads shine in high-concurrency, I/O-bound applications. They are particularly useful when handling large numbers of concurrent user requests or working with existing synchronous systems. With virtual threads, we can seamlessly replace traditional OS threads without rewriting code to use non-blocking APIs.

Looking ahead, I believe reactive programming and virtual threads will coexist for the foreseeable future. Over time, we might see some convergence between the two. For example, reactive stream libraries could evolve to leverage virtual threads, enabling developers to write reactive code that is both simpler and more efficient. Similarly, structured concurrency principles may influence reactive programming models, making them more robust and maintainable.

We're already seeing the first signs of this convergence. Vert.x 4.5 has begun experimenting with virtual threads, hinting at a future where these two paradigms seamlessly complement each other.

In the long run, virtual threads may become the default choice for many applications due to their simplicity and compatibility with existing synchronous code. The ability to write concurrent programs using familiar

imperative programming techniques is a significant advantage; it lowers the barrier to entry and reduces developers' cognitive overhead.

Only time will tell, but one thing is certain: concurrency in Java is evolving, and we, as developers, must evolve with it.